

Reviewer Pack — Minimal Frobenius Schur Index and Communication Complexity L...

Entry f1ccfad2bac3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 14:57:04 UTC

Minimal Frobenius Schur Index and Communication Complexity Lower Bound

Entry ID: f1ccfad2bac3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 14:57:04 UTC

1. Conjecture

Field A (mathematical branch): Algebra (Frobenius-Schur Indicators) **Field B** (complexity object): Communication Complexity

Statement:

For all Boolean functions f with n inputs, the minimal Frobenius-Schur index ($FSI_{\min}$) of any matrix representation of f is linearly correlated with the communication complexity lower bound (CC_{lower}) for f , such that $FSI_{\min} = \Omega(CC_{\text{lower}})$.

Rationale (proposer's reasoning):

The Frobenius-Schur indicator captures noncommutative properties of matrices and can reveal deep algebraic structure in computational problems. Communication complexity measures the minimal amount of communication needed to compute a function. This conjecture suggests that the algebraic structure revealed by $FSI_{\min}$ may directly relate to the fundamental limitations of communication.

Taxonomy category: Frobenius_Schur (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 64cd3cac0985e156

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient from a linear regression analysis of $FSI_{\min}$ against CC_{lower} is ≥ 0.8 , and no seed produces an $FSI_{\min} > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Frobenius-Schur index" AND "communication complexity" - "matrix representation" AND Boolean functions AND communication complexity" - "minimal Frobenius-Schur index" ~ "communication complexity lower bound"

Top relevant hits considered: - [http://arxiv.org/abs/1003.2698v1] The Pseudo-Hyperbolic Functions and the Matrix Representation of Eisenstein Complex Numbers - [http://arxiv.org/abs/1210.7842v2] Boolean Differential Operators - [http://arxiv.org/abs/2510.05045v2] A new Boolean matrix representation for Catalan semirings - [http://arxiv.org/abs/2001.04131v1] Observation of a resonant structure in $e^+e^- \rightarrow K^+K^-\pi^0\pi^0$ - [http://arxiv.org/abs/cs/9910010v2] Communication Complexity Lower Bounds by Polynomials - [http://arxiv.org/abs/1811.10525v1] Quantum Log-Approximate-Rank Conjecture is also False - [s2:86e01dd309a8c2254589634f2e9d872e89ca26f4] Optimal-Round Preprocessing-MPC via Polynomial Representation and Distributed Random Matrix (extended abstract) - [s2:10.1007/3-540-45294-X_15] Relations Between Communication Complexity, Linear Arrangements, and Computational Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=124, elapsed=240.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def matrix_representation(f):
        n = int(math.log2(len(f)))
        M = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                if f[i] == f[j]:
                    M[i][j] = 1
        return M

    def frobenius_schur_index(M):
        n = len(M)
        trace = sum(M[i][i] for i in range(n))
        det = determinant(M, n)
        return abs(trace / det)
```

```

def determinant(matrix, size):
    if size == 1:
        return matrix[0][0]
    det = 0
    for c in range(size):
        submatrix = [row[:c] + row[c+1:] for row in matrix[1:]]
        sign = (-1) ** (c % 2)
        det += sign * matrix[0][c] * determinant(submatrix, size - 1)
    return det

def communication_complexity(f):
    n = int(math.log2(len(f)))
    max_communication = 0
    for i in range(2**n):
        for j in range(2**n):
            if f[i] != f[j]:
                communication = bin(i ^ j).count('1')
                if communication > max_communication:
                    max_communication = communication
    return max_communication

n_values = [5, 10, 15, 20, 30, 40]
FSI_min_total = 0
CC_lower_total = 0
instances_tested = 0

for n in n_values:
    f = generate_boolean_function(n)
    M = matrix_representation(f)
    FSI_min = frobenius_schur_index(M)
    CC_lower = communication_complexity(f)

    if FSI_min > 10:
        return {
            "metric_name": "FSI_min",
            "metric_value": FSI_min,
            "instances_tested": instances_tested,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "FSI_min > 10"
        }

    FSI_min_total += FSI_min
    CC_lower_total += CC_lower
    instances_tested += len(f)

mean_FSI_min = FSI_min_total / instances_tested
mean_CC_lower = CC_lower_total / instances_tested

return {
    "metric_name": "FSI_min",
    "metric_value": mean_FSI_min,
    "instances_tested": instances_tested,
    "n_max": max(n_values),
    "conjecture_holds": mean_FSI_min >= 0.8 * mean_CC_lower,
}

```

```

        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 3 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_FSI_min = sum(r["metric_value"] for r in results) / len(results)
    std_FSI_min = math.sqrt(sum((r["metric_value"] - mean_FSI_min)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_FSI_min} std={std_FSI_min} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_FSI_min} std={std_FSI_min} support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"FSI_min > 10\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the correlation coefficient could not be calculated to determine if it meets the support condition of ≥ 0.8 . | next: Retry the experiment with a longer time limit or an alternative method to ensure the computation can complete and produce results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13526
2	preregistration	ollama_remote	glm4:latest	0	0	9589
3	novelty	ollama_remote	glm4:latest	0	0	8947
4	novelty	ollama_remote	glm4:latest	0	0	9746
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	60743
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11570
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10991
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	38389
9	judge	ollama_remote	glm4:latest	0	0	61834

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 225334 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/f1ccfad2bac3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f1ccfad2bac3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f1ccfad2bac3.tar.gz` (if generated)